

Introducción a Objetos en JavaScript

¿Qué es un Objeto?

Los objetos son fundamentales en JavaScript y representan estructuras de datos que agrupan información relacionada. En esencia, un objeto es un conjunto de pares clave-valor, donde la clave es una cadena que actúa como un identificador y el valor puede ser de cualquier tipo, incluyendo números, cadenas, booleanos, funciones e incluso otros objetos. Esta flexibilidad permite a los desarrolladores modelar datos del mundo real de manera más efectiva.

Los objetos permiten organizar y encapsular información, lo que resulta en un código más limpio y mantenible. Por ejemplo, en lugar de tener múltiples variables para representar las propiedades de un estudiante (nombre, edad, calificaciones), podemos encapsular toda esta información dentro de un solo objeto, facilitando su gestión.

Creación de Objetos

Existen diversas maneras de crear objetos en JavaScript, exploramos a continuación 3 maneras principales para este propósito.

a. Sintaxis de objeto literal. La forma más común de crear un objeto es mediante la sintaxis de objeto literal, que consiste en encerrar una lista de propiedades entre llaves. Este método es intuitivo y directo, lo que lo convierte en la opción preferida para la creación de objetos simples y estáticos. Se pueden definir propiedades de manera explícita y clara, lo que facilita la lectura del código.

Además, los objetos literales pueden incluir métodos, que son funciones que se pueden invocar como parte del objeto. Esto permite encapsular tanto datos como comportamiento en una sola entidad. Por ejemplo, un objeto persona no solo puede tener propiedades como nombre y edad, sino que también puede incluir un método que permita a la persona presentarse.

Ejemplo:

```
const persona = {
```

```
  nombre: "Juan",
```

edad: 30,

ocupacion: "Desarrollador"

};

b. Usando el constructor Object. Otra manera de crear objetos es usando el constructor Object. Esta técnica es útil en situaciones donde se necesita más flexibilidad, como al crear objetos dinámicamente. A través del constructor, podemos crear un objeto vacío y luego añadirle propiedades y métodos de forma programática. Esto es especialmente valioso en aplicaciones más complejas donde los objetos pueden requerir una configuración más elaborada.

Usar el constructor Object también puede facilitar la creación de objetos en bucles o cuando se trabaja con datos que provienen de fuentes externas, como API. Esto proporciona un enfoque más programático para definir objetos en situaciones donde la estructura puede no ser conocida de antemano.

Ejemplo:

```
const coche = new Object();
```

```
coche.marca = "Toyota";
```

```
coche.modelo = "Corolla";
```

```
coche.año = 2020;
```

c. Usando clases (ES6). La introducción de clases en ES6 proporciona una sintaxis más estructurada y legible para crear objetos. Al definir una clase, podemos establecer un modelo para crear múltiples instancias del mismo tipo de objeto, cada una con sus propias propiedades y métodos. Esto promueve la reutilización del código y la organización.

Las clases permiten también la implementación de la herencia, lo que significa que podemos crear clases derivadas que heredan características de una clase base. Esto

es especialmente útil para organizar objetos que comparten ciertas propiedades o métodos, permitiendo un enfoque más modular y escalable.

Ejemplo:

```
class Animal {  
  
  constructor(especie, sonido) {  
  
    this.especie = especie;  
  
    this.sonido = sonido;  
  
  }  
  
  hacerSonido() {  
  
    console.log(this.sonido);  
  
  }  
  
}
```

```
const perro = new Animal("Perro", "Guau");
```

Se verá con mayor profundidad este tema en módulos sucesivos del Máster.

Acceso a Propiedades

Puedes acceder a las propiedades de un objeto de dos maneras:

a. Notación de punto. Acceder a las propiedades de un objeto es fundamental para manipular sus datos. La notación de punto es la forma más común de hacerlo, ya que

es directa y fácil de leer. Al usar la notación de punto, simplemente especificamos el nombre del objeto seguido de un punto y el nombre de la propiedad. Esto es particularmente útil en situaciones donde sabemos de antemano el nombre de las propiedades.

Ejemplo:

```
console.log(persona.nombre); // "Juan"
```

b. Notación de corchetes. Por otro lado, la notación de corchetes es igualmente válida y puede ser más conveniente en ciertos casos. Por ejemplo, si el nombre de la propiedad es dinámico o contiene espacios, la notación de corchetes nos permite acceder a estas propiedades usando cadenas. Esta flexibilidad es crucial al trabajar con datos que pueden variar, como en aplicaciones que manipulan formularios o datos de usuarios.

Ejemplo:

```
console.log(persona["edad"]); // 30
```

Modificación de Propiedades

Puedes cambiar el valor de una propiedad existente o agregar nuevas propiedades.

Modificar las propiedades de un objeto es un proceso sencillo y directo. Podemos actualizar el valor de una propiedad existente simplemente asignándole un nuevo valor utilizando la notación de punto o de corchetes. Esta capacidad de modificación en tiempo de ejecución es una de las razones por las que los objetos son tan poderosos en JavaScript; permiten que los datos sean dinámicos y adaptativos.

Además, agregar nuevas propiedades a un objeto es igual de sencillo. Esto significa que podemos ampliar un objeto a medida que nuestra aplicación evoluciona, sin necesidad de redefinir el objeto por completo. Esta característica hace que los objetos sean ideales para manejar datos en situaciones en las que la estructura puede cambiar o crecer con el tiempo.

Ejemplo:

```
persona.edad = 31; // Modificar
```

```
persona.email = "juan@example.com"; // Agregar
```

Métodos de Objetos

Un **método es una función que se asocia a un objeto**. Los métodos además pueden operar sobre las propiedades de los objetos. La creación de métodos dentro de un objeto no solo permite encapsular el comportamiento relacionado con ese objeto, sino que también promueve un diseño orientado a objetos más limpio. Por ejemplo, un objeto calculadora puede contener métodos para sumar, restar, multiplicar y dividir, centralizando toda la lógica matemática relacionada en un solo lugar.

Al implementar métodos, podemos manipular directamente las propiedades del objeto, lo que permite un mayor control sobre los datos. Esto es especialmente útil en aplicaciones donde se requiere que las propiedades cambien en respuesta a las acciones del usuario o eventos en la aplicación.

Ejemplo:

```
const calculadora = {  
  sumar: function(a, b) {  
    return a + b;  
  },  
  restar: function(a, b) {  
    return a - b;  
  }  
};
```

```
console.log(calculadora.sumar(5, 3)); // 8
```

Ejercicios para practicar

1. Crear un objeto "auto" con propiedades como marca, modelo, año, y un método mostrarInfo que imprima la información del auto.
2. Modelar cualquier otro objeto del mundo real que prefieras con JS de manera similar.